



Scaling Robotics to 200 Developers

Flexible C++ Architectures for Kinematics,
Dynamics, Simulation & Control

CARL SALDANHA

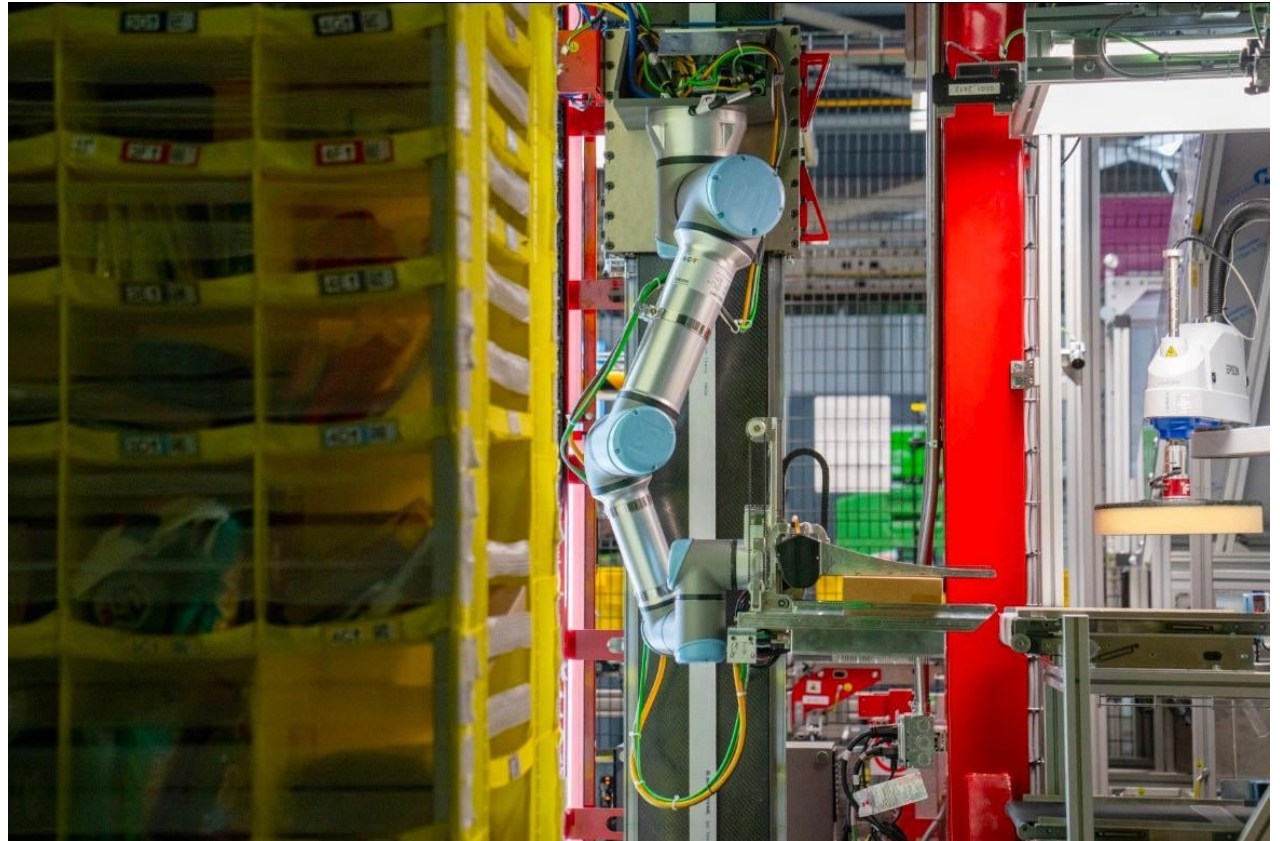


20
26



What do I work on?

- Manipulation Robotics Organization
- Our group is focused on manipulation robotics of various kinds
- We help solve various problems inside Amazon's warehouses from picking; packing and sorting

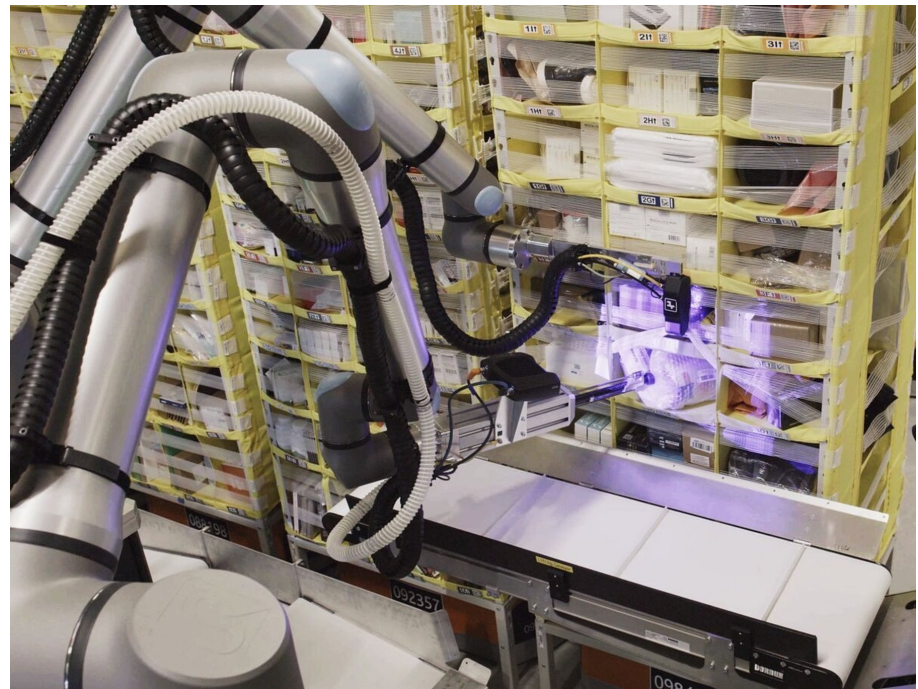




The problems of scale are many ...

Biggest projects have over 10 billion packages moved while the smaller ones still have millions

Our focus today will be discussing interface design for robotics algorithms.



Our Challenges

- As we develop new projects and ideas how do we create robust interfaces
- As the field continues innovating how do we make sure we have a set of interfaces that can absorb those innovations



Our Challenges

- As we develop new programs and ideas how do we create robust interfaces

- As the field continues innovating how do we make sure we have a set of interfaces that can absorb those innovations



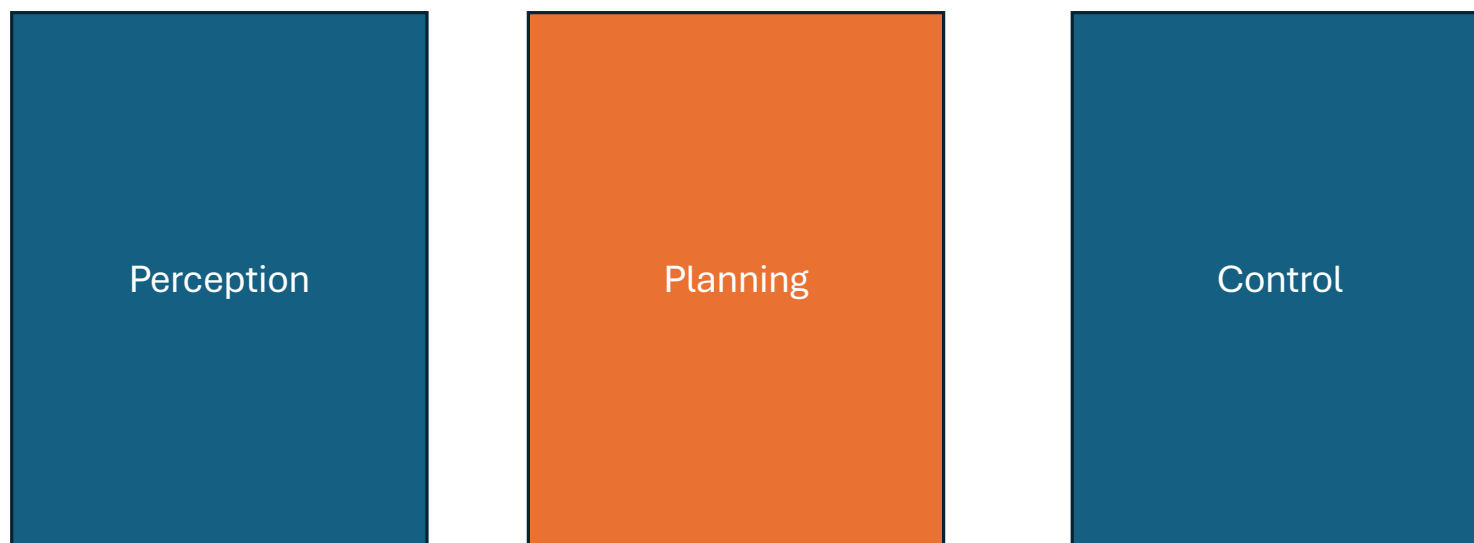
Notes For This Talk

- The majority of this stack was built for robotic manipulators so there's a lot of specific machinery for that
- This interfaces in this talk are used to run controllers at up to 1KHz but in a high level non-safety critical system.
- While performant; these shouldn't be considered safe
- These also encourage use of third party code which is hard to justify in safety critical applications

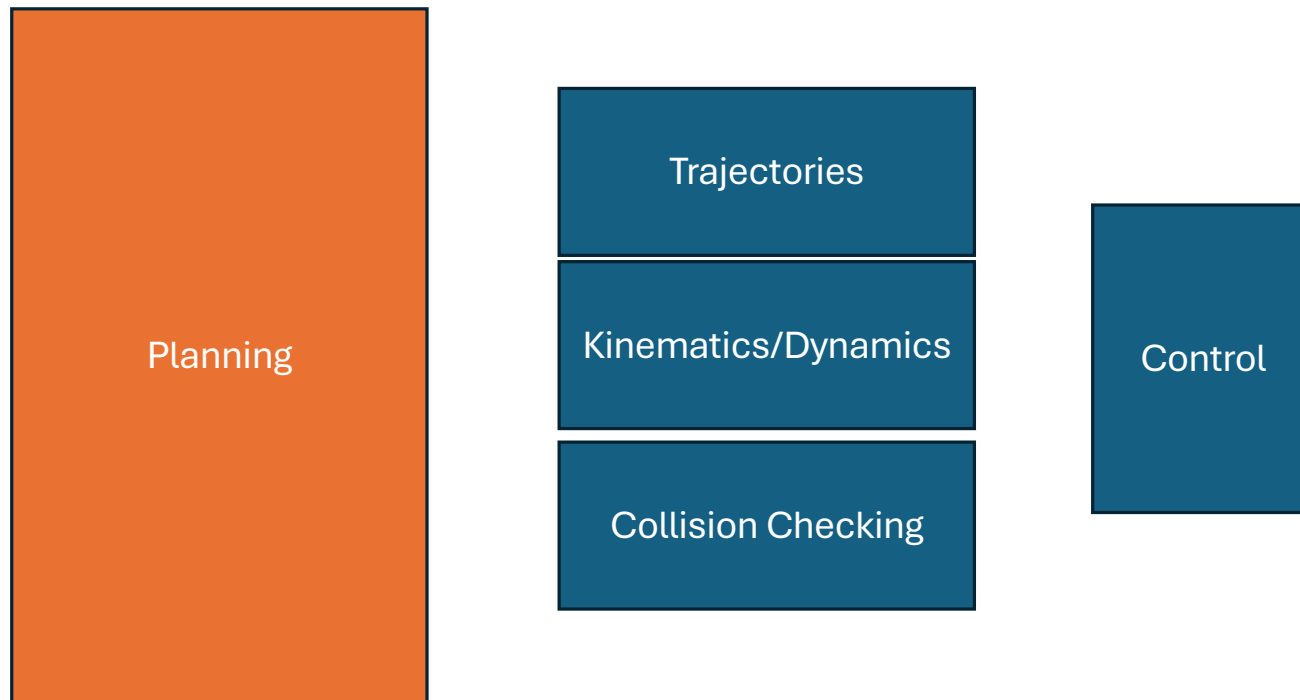
Canonical Areas of Robotics Stacks



Majority of the Focus For this Talk



Majority of the Focus For this Talk



Goals for Interfaces in Planning

- Never change the interface
- The interfaces definitions shouldn't hinder performance
- As libraries change; we can test and absorb their changes
- Make it easy to read and maintain

Goals for Interfaces in Planning

- **Robust:** Never change the interface
- **Performant:** The interfaces definitions shouldn't hinder performance
- **Extensible:** As libraries change; we can test and absorb their changes
- **Modern readable C++ :** Make it easy to read and maintain

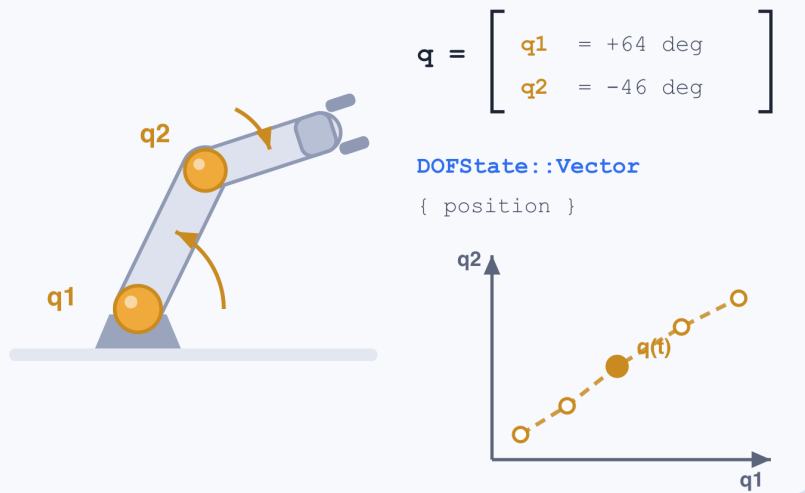
Goals for Interfaces in Planning

- **Robust**
- **Performant**
- **Extensible**
- **Modern readable C++** (with Python Bindings)

Definitions

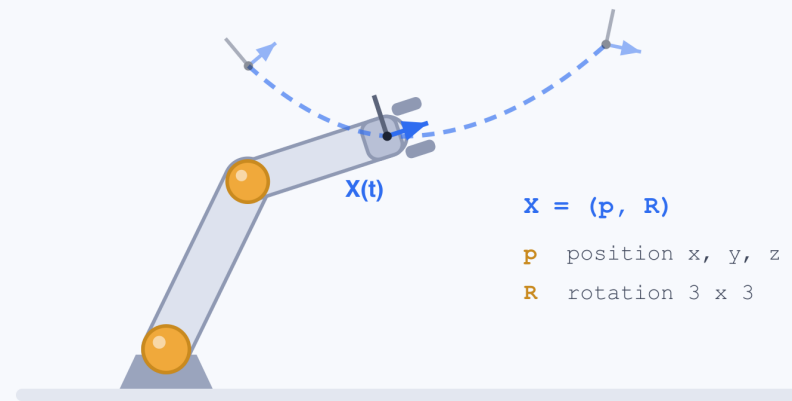
JOINT SPACE

one angle per joint $\rightarrow q$ in $\mathbb{R}^n$



TASK SPACE

one pose for the end-effector $\rightarrow X$ in $SE(3)$



JSReference

`{ pos, vel, acc, jrk }`

one of each PER JOINT: position, velocity, acceleration, jerk

TSReference

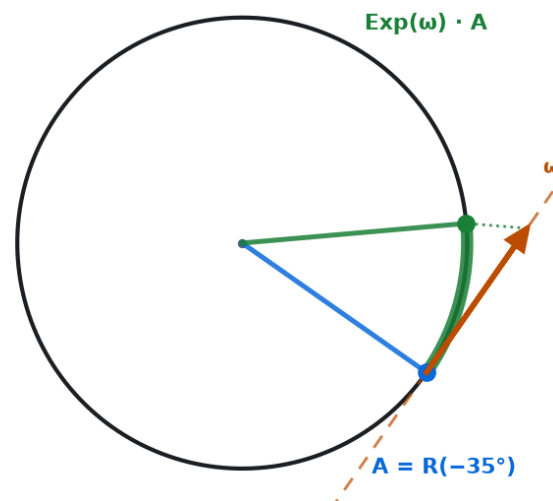
`{ pos: Pose3, vel, acc }`

an $SE(3)$ pose + spatial velocity / acceleration (twists)

Trajectories

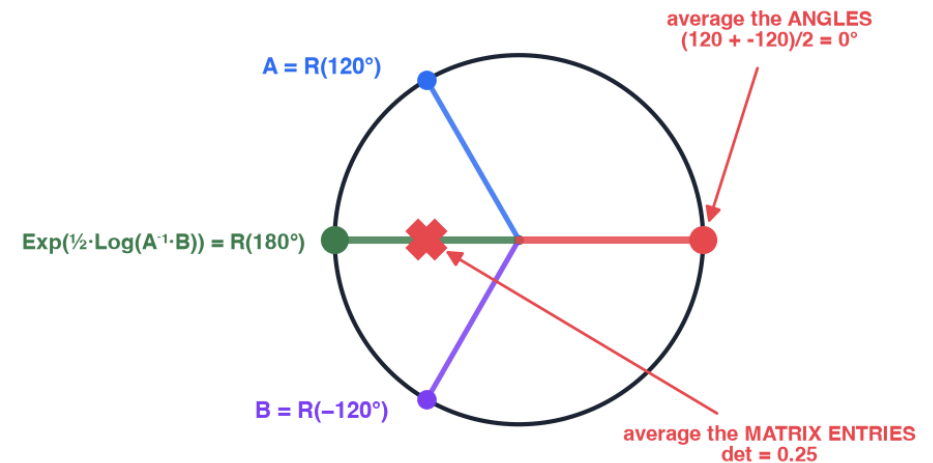
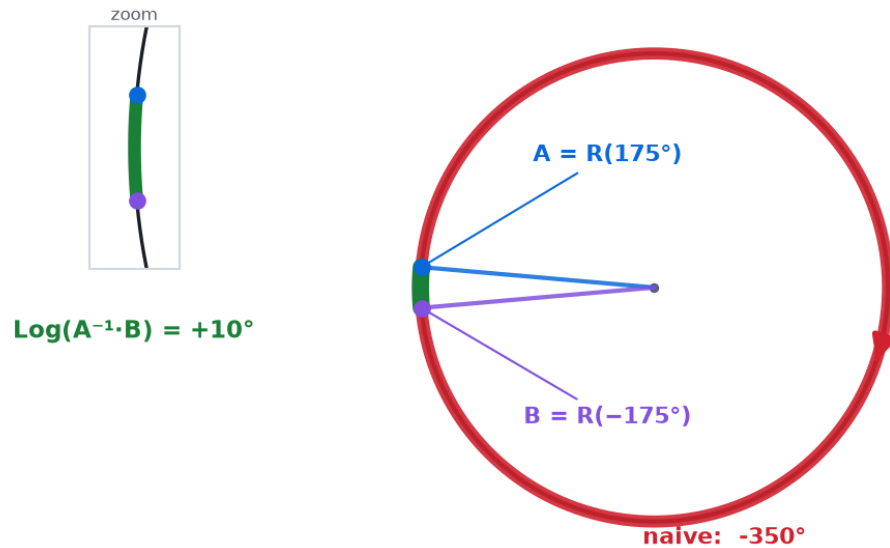
A (very) brief primer on Lie Groups

- Lie groups are smooth manifolds that have useful properties in robotics.
- Example below is 2d rotations ($SO(2)$). Exponent maps a tangent vector to a group element (moving you by ω for a unit time)



A (very) brief primer on Lie Groups

- Some nice properties are that the difference across the group always give you the geodesic (shortest) path
- It allows you to add two points together while guaranteeing the answer is in the group



Lie Groups Implementation

- Our goal is to construct SE3 which is rotations and positions in 3D
- We start with a traits class that has the definitions for points and dimensions we need

```
// No default definition: every Lie group must specialize.
template <class Derived>
struct LieGroupTraits;

template <class Scalar>
struct LieGroupTraits<SE3<Scalar>> {
    using Point      = SE3Point<Scalar>;
    // . . .
    static constexpr int GroupDim  = 6;
    static constexpr int MatrixDim = 4;
};
```

Lie Groups Implementation

```
// The base recovers the concrete types from Derived.
template <class Derived>
class LieGroupBase {
    using Traits = LieGroupTraits<Derived>;

public:
    using Point      = typename Traits::Point;
    static constexpr int GroupDim  = Traits::GroupDim;
    static constexpr int MatrixDim = Traits::MatrixDim;

    // Compile-time forwarding to group-specific math
    static Point Exp(const Vector& xi) {
        return Derived::Exp(xi);
    }

    // Shared algorithms are inherited by every group.
    static Vector LeftError(const Point& x1, const Point& x2) {
        return Derived::Log(Between(x1, x2));
    }

    static Point Interpolate(
        Scalar alpha, const Point& x1, const Point& x2) {
        return x1 * Exp(alpha * LeftError(x1, x2));
    }
};
```

Lie Groups Implementation

```
template <typename Scalar>
class SE3 : public LieGroupBase<SE3<Scalar>> {
public:
    using Point = typename LieGroupTraits<SE3<Scalar>>::Point;
    using Vector = Eigen::Matrix<Scalar, 6, 1>;

    // Tangent -> pose: xi = [phi; rho]
    static Point Exp(const Vector& xi) {
        Point output(
            SO3<Scalar>::Exp(xi.template head<3>()),
            SO3<Scalar>::LeftJacobian(
                xi.template head<3>()) * xi.template tail<3>());

        return output;
    }

    // Interpolate(), Between(), LeftError(), ... are inherited.
};
```

Lie Groups Implementation

```
template <typename Scalar>
class SE3 : public LieGroupBase<SE3<Scalar>> {
public:
    using Point = typename LieGroupTraits<SE3<Scalar>>::Point;
    using Vector = Eigen::Matrix<Scalar, 6, 1>;

    // Tangent -> pose: xi = [phi; rho]
    static Point Exp(const Vector& xi) {
        Point output(
            SO3<Scalar>::Exp(xi.template head<3>()),
            SO3<Scalar>::LeftJacobian(
                xi.template head<3>()) * xi.template tail<3>());

        return output;
    }

    // Interpolate(), Between(), LeftError(), ... are inherited.
};

template <typename Scalar>
class SE3Point : public LieGroupPointBase<SE3<Scalar>> {
private:
    SO3Point<Scalar> m_orientation; // unit quaternion
    TPoint<Scalar, 3> m_position;    // translation
};
```

C RTP
Implementation

Motions!

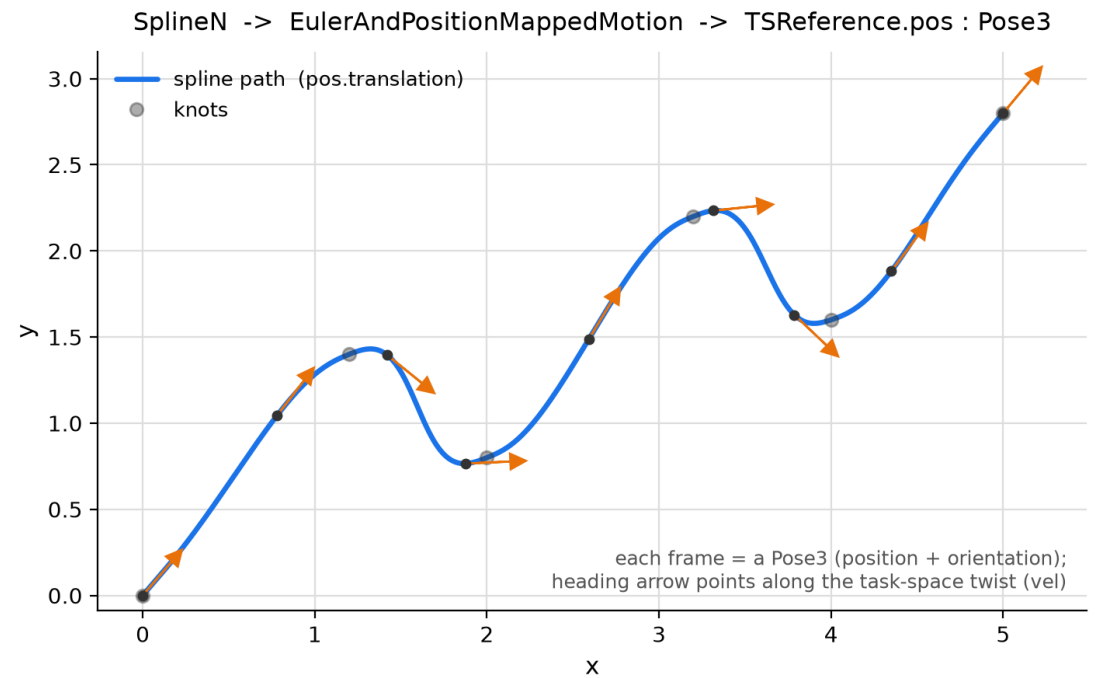
```
template <typename LieGroup>
class Motion {
public:

    // ---- evaluate: point = f(s) ----
    virtual void evaluate_at(double phase, LieGroup
::Point* point) const = 0;

    // finite-difference ON THE MANIFOLD (via
    Manifold::local == Log(a-1◦b)).
    VectorX numerical_derivative_at(double phase,
    int order = 1,
                                double delta =
    1e-6) const;

    // ---- shape / domain metadata ----
    virtual double get_duration() const = 0;
    // phase length

    // ....
};
```



Motions!

```
template <Rn>
class JSTrajectory<Rn> {
public:

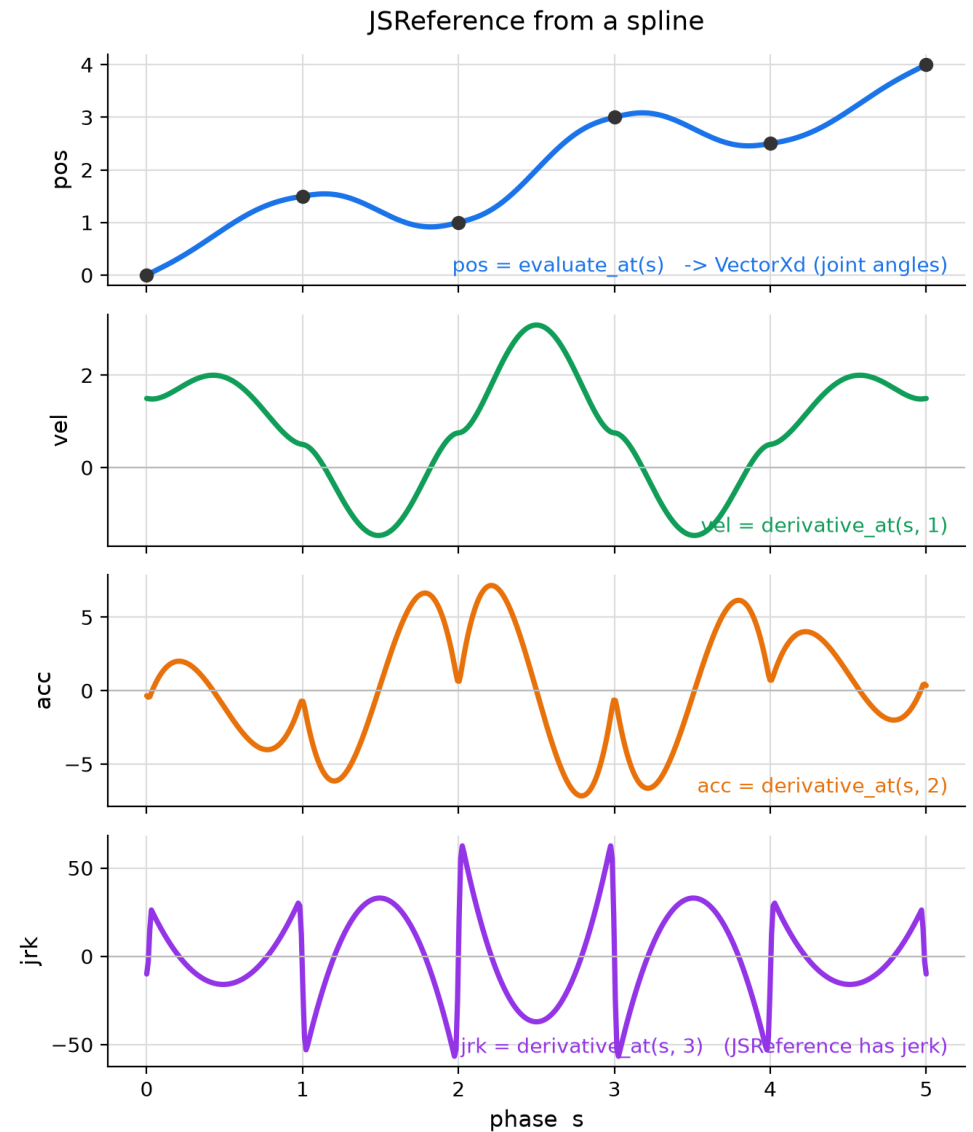
    // ---- evaluate: point = f(s) ----
    virtual void evaluate_at(double s,
        ManifoldPoint* point) const = 0;

    // ---- derivatives ( $\mathbb{R}^n$  tangent vectors) ----
    // k-th derivative at s; default delegates to
    numerical_derivative_at.
    virtual VectorX derivative_at(double s, int
        order = 1) const;

    // finite-difference ON THE MANIFOLD (via
    Manifold::local == Log(a-1◦b)).
    VectorX numerical_derivative_at(double s, int
        order = 1,
        double delta =
        1e-6) const;

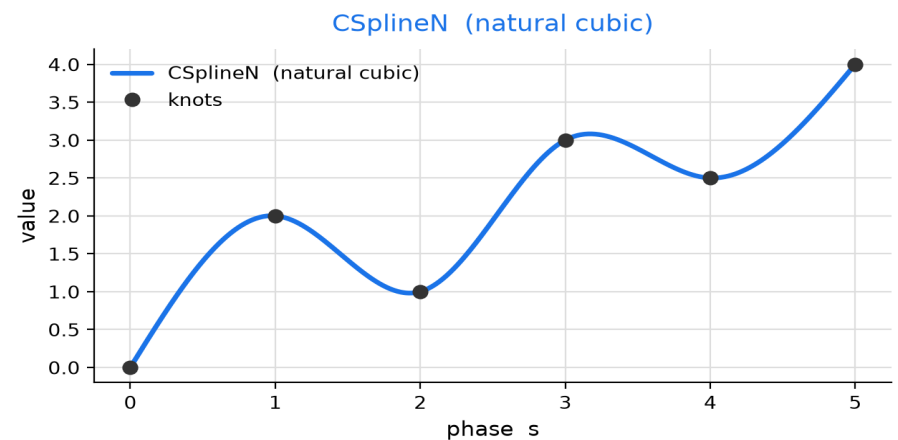
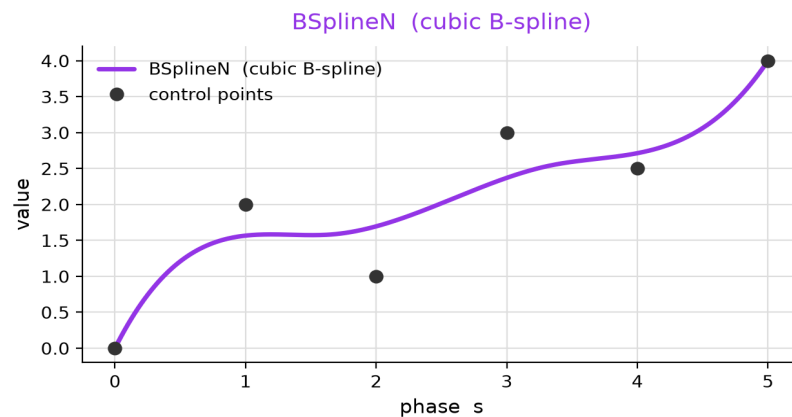
    // ---- shape / domain metadata ----
    virtual double get_duration() const = 0;
    // phase length

    // ....
}
```



Definitions for Movement

```
class SplineN {  
  
    // — phase domain —  
    virtual double get_minimum_phase() const = 0;  
    virtual double get_maximum_phase() const = 0;  
    double        get_duration()      const;  
  
    // — evaluate (by-value + realtime no-alloc) —  
    virtual void   evaluate_at(double s, VectorXd& out) const = 0;    // no-alloc  
  
    // — derivatives (by-value + realtime no-alloc) —  
    virtual void   derivative_at(double s, int derivative_order, VectorXd& out) const = 0;  
  
};
```



Do these interfaces meet our criteria?

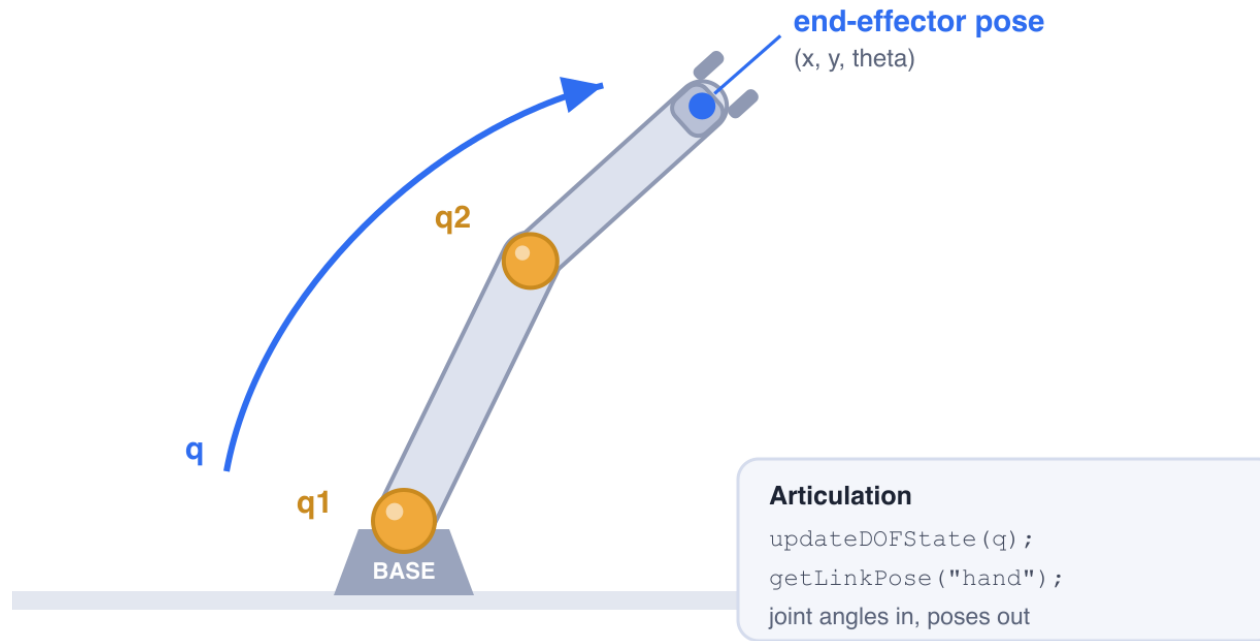
- **Robust:** The core interfaces haven't changed in a long time
- **Performant:** Every path in the core library is carefully checked for allocations
- **Extensible:** Without major changes we have integrated Ruckig motions, new spline types, new manifolds (SO3xR3)
- **Modern readable C++** (with Python Bindings): Yes

Now we have concrete types constructing poses and motions

Now we have concrete types constructing poses and motions
lets use them

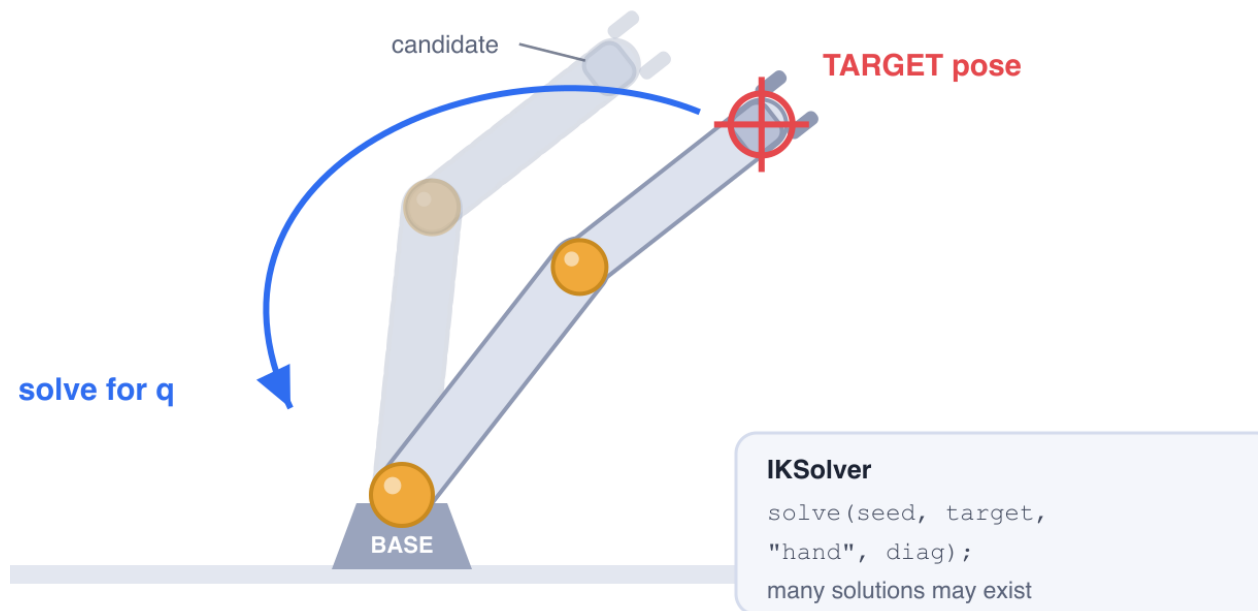
Kinematics & Dynamics

Kinodynamics



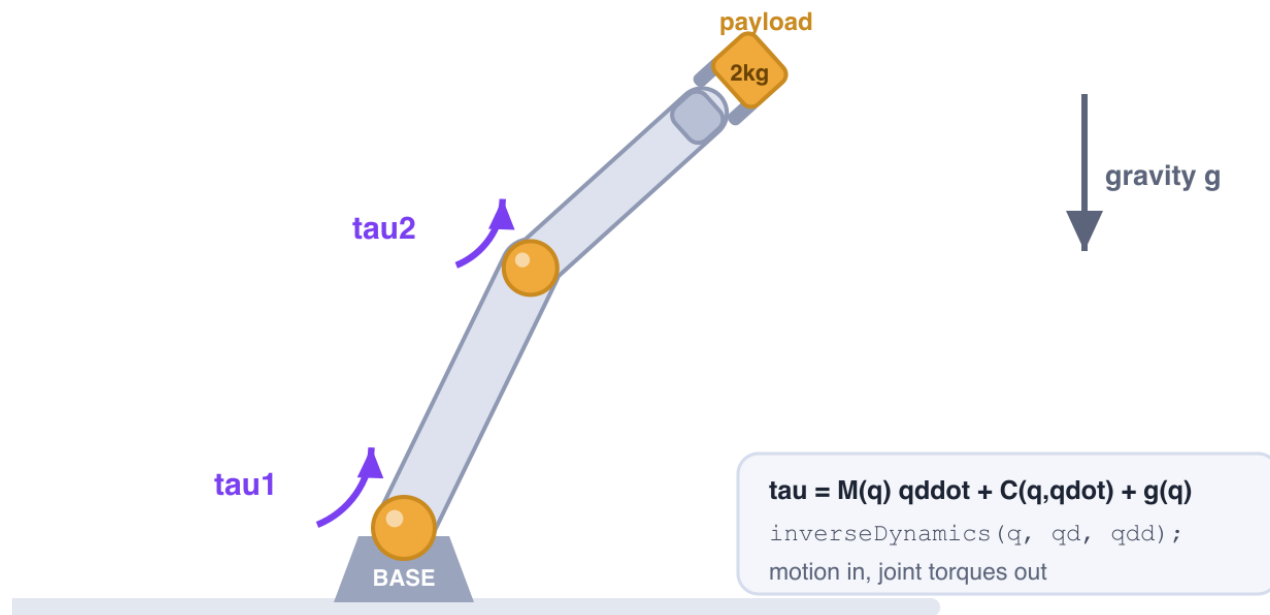
Forward Kinematics

Kinodynamics



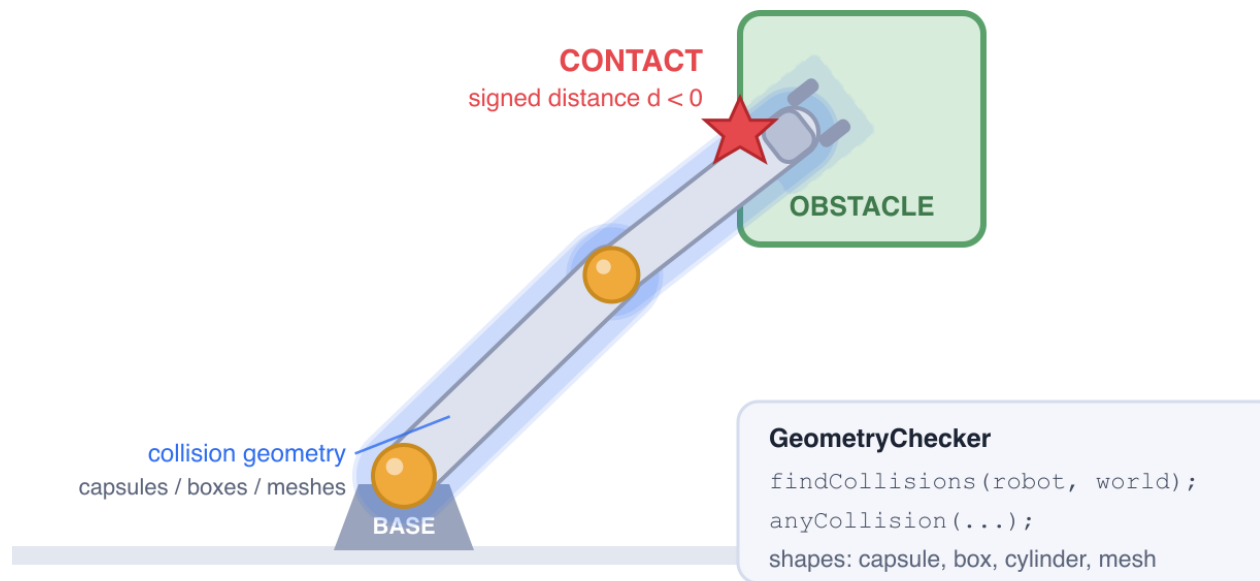
Inverse Kinematics

Kinodynamics



Inverse Dynamics

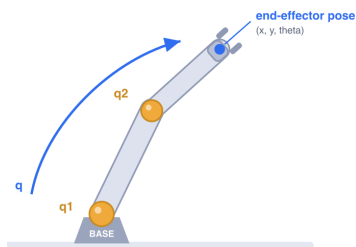
Kinodynamics



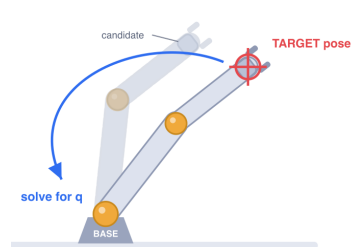
Collision Checking

Kinodynamics

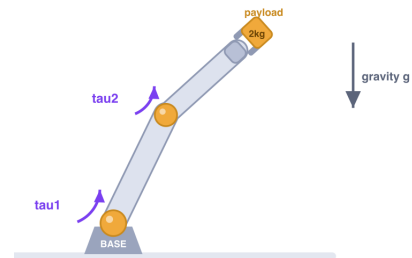
Forward Kinematics



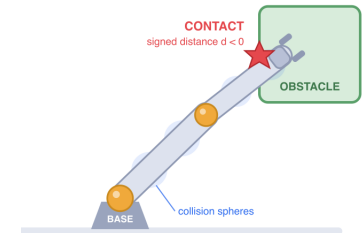
Inverse Kinematics



Inverse Dynamics

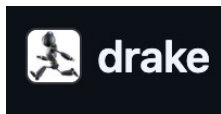


Collision Checking



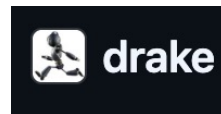
Kinodynamics

Forward Kinematics

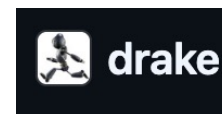


Inverse Kinematics

IKFast



Inverse Dynamics



Collision Checking



Coal

Articulations

```
class Articulation {
public:
    using Handle = std::shared_ptr<Articulation>;
    virtual ~Articulation() = default;

    // ---- build (cold path; allocation here is fine) ----
    virtual std::shared_ptr<Link> getRootLink() const = 0;
    virtual std::shared_ptr<Link> addLink(const Link& link,
                                         const std::shared_ptr<Link>& parent) = 0;

    // ---- DOF indexing (scalar returns - no allocation) ----
    virtual size_t getDimension() const = 0;

    // ---- set configuration + run FK (takes a view; allocates nothing) ----
    virtual void updateDOFState(std::span<const DOFState> dofState) = 0;

    // ---- read configuration into a caller-owned buffer ----
    //   out is resized once by the caller (out.resize(getDimension())) then reused.
    virtual void getDOFState(std::span<DOFState> out) const = 0;

    // ---- FK: fill a caller-owned Pose3 (Pose3 is a value type, no heap) ----
    virtual void getLinkPose(std::string_view linkName, Pose3& out) const = 0;

    // ---- Jacobian: fill a caller-owned 6xN matrix, no allocation on the hot path ----
    //   Caller sizes once: J.resize(6, getDimension()); the impl writes in place.
    virtual void getLinkJacobian(
        std::string_view linkName,
        Eigen::Ref<Eigen::MatrixXf> J) const = 0;
};
```

Internal
Representation

+



Articulations

```
using namespace kinodynamics::core;
using kinodynamics::playground::buildAbbIrb4600_40_255;

// The planner/solver never names a backend - it takes the abstract handle.
void planTo(Articulation::Handle robot, const Pose3& goal) {
    OptimizationIKSolver::Parameters params;
    auto solver = OptimizationIKSolver::create(std::move(robot), params);
    // ... solver->solve(goal) ...
}

// Same robot, same planner. One enum value picks the physics backend.
int main() {
    Pose3 goal = /* ... */;

    // PhysX backend -> PhysxArticulation under the hood
    Articulation::Handle px = buildAbbIrb4600_40_255(PhysicsBackend::PhysX);
    planTo(px, goal);

    // Swap the backend: Pinocchio -> PinocchioArticulation. Planner code is unchanged.
    Articulation::Handle pin = buildAbbIrb4600_40_255(PhysicsBackend::Pinocchio);
    planTo(pin, goal);
}
```



Inverse Kinematics

IK Fast
+
Jacobian
Pseudoinverse

```
class IKSolver {  
    explicit IKSolver(Articulation::Handle articulation);  
  
    class Parameters {  
    public:  
        float rotTolerance = DEFAULT_ROT_TOLERANCE;  
        float posTolerance = DEFAULT_POS_TOLERANCE;  
        Pose3 offsetTransform = Transform3();  
        std::span<const std::string_view> solveJoints{};  
        // ...  
    };  
  
    struct Solution {  
        DOFState::Vector dofs;  
        bool solverSingularity;  
        bool allGoalsSatisfied;  
    };  
  
    // — solve: fills a caller-owned buffer; zero allocation on the hot path —  
    // caller reserves `out` once and reuses it across solves (capacity retained).  
    // returns the number of solutions written.  
    [[nodiscard]] virtual size_t solve(  
        std::span<const DOFState> seed,  
        const Pose3& targetPose,  
        std::string_view linkName,  
        std::vector<Solution>& out,           // cleared + refilled in place  
        Diagnostics& diagnostics) = 0;  
  
    virtual void setParameters(const Parameters&) = 0;  
  
    // ...  
};
```

Inverse Kinematics

IK Fast
+
Jacobian
Pseudoinverse
+
Optimization

```
class IKSolver {
    struct Solution {
        DOFState::Vector dofs;
        bool solverSingularity;
        bool allGoalsSatisfied;
    };

    // — solve: fills a caller-owned buffer; zero allocation on the hot path —
    //   caller reserves `out` once and reuses it across solves (capacity retained).
    //   returns the number of solutions written.
    [[nodiscard]] virtual size_t solve(
        std::span<const DOFState> seed,
        const Pose3& targetPose,
        std::string_view linkName,
        std::vector<Solution>& out, // cleared + refilled in place
        Diagnostics& diagnostics) = 0;

    virtual void setParameters(const Parameters&) = 0;

    [[nodiscard]] virtual ConstraintSupport addConstraint(const Constraint&) = 0;

    // Ask BEFORE adding, so callers can branch without triggering a rejection.
    [[nodiscard]] virtual ConstraintSupport supports(const Constraint&) const = 0;

};
```

Do these interfaces meet our criteria?

- **Robust:** Upgrades from Internal Implementations through PhysX; Pinocchio & Dart have been successful
- **Performant:** Kinematics has never been a performance bottleneck with uS performance on IK and FK on fairly large articulations
- **Extensible:** Have made serious extensions to joint types with mimic and curvilinear joint support with no major changes
- **Modern readable C++:** The C++ here should probably be updated to doing its choices at compile time

Collision Checking

Checking Geometry

```
class GeometryChecker {
public:
    virtual bool anyCollision(
        const GeometricBodyGroup& a,
        const GeometricBodyGroup& b,
        const CollisionCheckParams& params,
        CollisionReport& hit) const = 0;

    virtual ~GeometryChecker() = default;
};
```

```
class GeometricBodyGroup {
public:

    // Static object at a world pose
    virtual void add(
        std::string bodyName,
        std::shared_ptr<Shape> shape,
        Transform3 world_H_shape = {}) = 0;

    // Dynamic articulated object
    virtual void add(
        std::string bodyName,
        Articulation::Handle articulation) = 0;

    // Environment geometry
    virtual void add(
        std::string bodyName,
        const octomap::OcTree& tree,
        const OctomapConversionOptions& = {}) = 0;

    virtual void remove(std::string bodyName) = 0;
    virtual Handle clone(std::string newName) const = 0;
};
```

Checking Geometry



Coal

```
class GeometryChecker {
public:
    virtual bool anyCollision(
        const GeometricBodyGroup& a,
        const GeometricBodyGroup& b,
        const CollisionCheckParams& params,
        CollisionReport& hit) const = 0;

    virtual ~GeometryChecker() = default;
};
```

```
class GeometricBodyGroup {
public:

    // Static object at a world pose
    virtual void add(
        std::string bodyName,
        std::shared_ptr<Shape> shape,
        Transform3 world_H_shape = {}) = 0;

    // Dynamic articulated object
    virtual void add(
        std::string bodyName,
        Articulation::Handle articulation) = 0;

    // Environment geometry
    virtual void add(
        std::string bodyName,
        const octomap::OcTree& tree,
        const OctomapConversionOptions& = {}) = 0;

    virtual void remove(std::string bodyName) = 0;
    virtual Handle clone(std::string newName) const = 0;
};
```

Checking Geometry



Coal

```
class GeometryChecker {
public:
    virtual bool anyCollision(
        const GeometricBodyGroup& a,
        const GeometricBodyGroup& b,
        const CollisionCheckParams& params,
        CollisionReport& hit) const = 0;

    virtual std::size_t queryProximity(
        const GeometricBodyGroup& a,
        const GeometricBodyGroup& b,
        const ProximityRequest& request,
        std::span<ProximityResult> results) const = 0;

    virtual ~GeometryChecker() = default;
};
```

```
class GeometricBodyGroup {
public:

    // Static object at a world pose
    virtual void add(
        std::string bodyName,
        std::shared_ptr<Shape> shape,
        Transform3 world_H_shape = {}) = 0;

    // Dynamic articulated object
    virtual void add(
        std::string bodyName,
        Articulation::Handle articulation) = 0;

    // Environment geometry
    virtual void add(
        std::string bodyName,
        const octomap::OcTree& tree,
        const OctomapConversionOptions& = {}) = 0;

    virtual void remove(std::string bodyName) = 0;
    virtual Handle clone(std::string newName) const = 0;
};
```

Do these interfaces meet our criteria?

- **Robust:** Not quite. The core interfaces have been through many iterations
- **Performant:** The library is the touchiest performance bottleneck and improving performance is a priority
- **Extensible:** Yes, We have integrated VAMP and GPU collision checking and these libraries work well with the older interfaces.
- **Modern readable C++** (with Python Bindings): Yes

Control

Controller Interfaces

```
class MyController : public Controller {
public:
    static bool is_supported(const RobotControllerConfig& c);
    ControllerType get_type() const override
        { return ControllerType::MY_LAW; }

    ControllerState initialize(const RobotControllerStatus& s,
                             const RobotControllerCommand& cmd) override;
    ControllerState update(const RobotControllerStatus& s,
                          DriverCommand& out) override // the law
    const std::vector<Part>& get_supported_parts() const override;
    // get_commanded_parts / state / fault: boilerplate
};
```

Joint Space Controller Interfaces

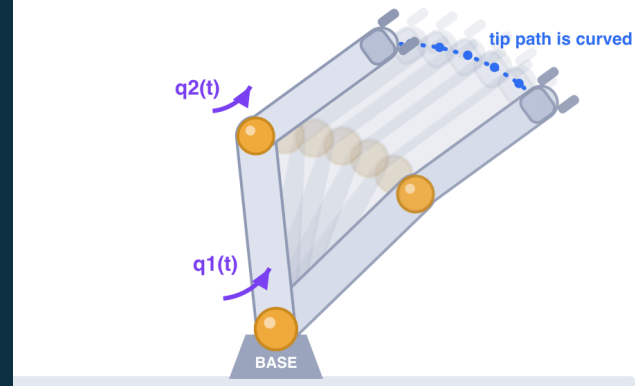
```
ControllerState JointSpaceController::update(const
RobotControllerStatus& status,
                                           DriverCommand& cmd) {
    const double s = seconds_since(m_start);           // trajectory phase

    // sample the Dilithium JSTrajectory (Motion<Rn>)
    m_ref.pos = m_traj.evaluate_at(s);
    m_ref.vel = m_traj.derivative_at(s, 1);
    m_ref.acc = m_traj.derivative_at(s, 2);

    // feedback
    m_fbk.pos = status.parts.at(Part::ARM).current_position;
    m_fbk.vel = status.parts.at(Part::ARM).current_velocity;

    // JS law (kp/kd/ki per DOF) → commanded acceleration, then integrate
    auto out = m_law.run(m_ref, m_fbk);
    m_cmd.acc = out.cmd.total;
    m_cmd.vel += m_dt * m_cmd.acc;
    m_cmd.pos += m_dt * m_cmd.vel;

    cmd.part_commands.at(Part::ARM)
        .set_streaming_command(m_cmd.pos, m_cmd.vel, m_cmd.acc,
/*torque=*/{});
    if (s >= m_traj.get_duration()) m_state = ControllerState::COMPLETE;
    return m_state;
}
```



Task Space Controller Interfaces

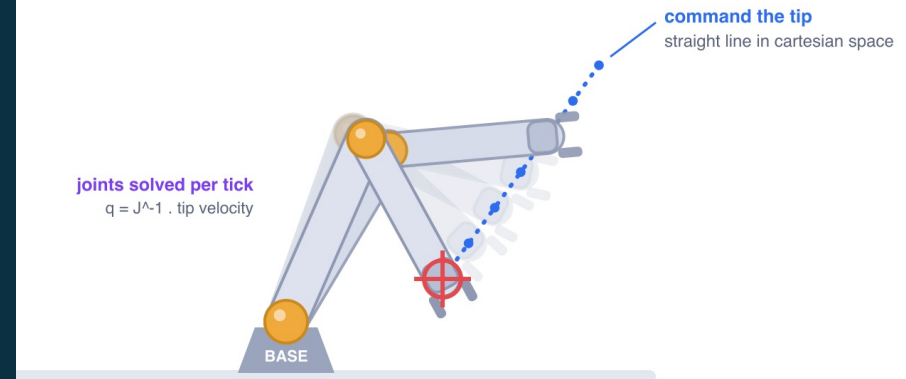
```
ControllerState TaskSpaceWaypointController::update(const
RobotControllerStatus& status,
                                                    DriverCommand&
cmd) {
    if (m_state != ControllerState::IN_MOTION) return m_state;
    const double s = seconds_since(m_start);          // trajectory
phase

    // sample the Dilithium TSTrajectory (Motion<SE3>) - desired task
twist
    SpatialVector3d V_des = m_pose_traj.derivative_at(s, 1);

    // Jacobian + damped pseudo-inverse - recomputed EVERY tick (DART-
backed)
    auto J      = m_model->get_jacobian(Part::ARM, m_frame_id);
    auto Jpinv  = m_model->get_jacobian_pinv(Part::ARM, m_frame_id,
m_pinv_damping); // DLS

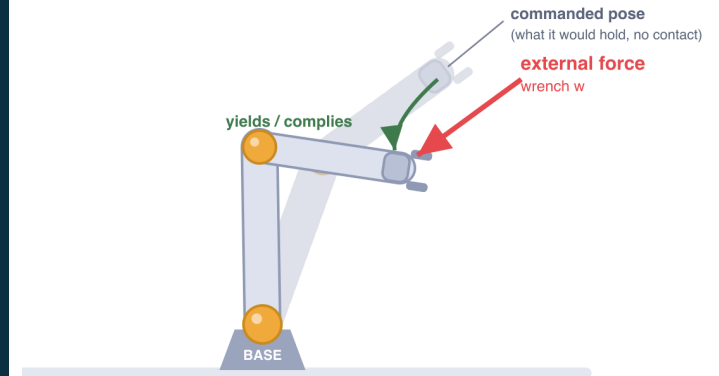
    // IK: map task twist → joint velocity, then integrate
    m_js.vel  = Jpinv * V_des;
    m_js.pos += m_dt * m_js.vel;

    cmd.part_commands.at(Part::ARM).set_streaming_command(m_js.pos,
m_js.vel, {}, {});
    if (m_supervisor.trajectory_complete(status)) m_state =
ControllerState::COMPLETE;
    return m_state;
}
```



Admittance Controller Interfaces

```
ControllerState TaskSpaceAdmittanceController::update(  
    const RobotControllerStatus& status,  
    DriverCommand& cmd) {  
    if (m_state != ControllerState::IN_MOTION) return m_state;  
    const double s = seconds_since(m_start);           // trajectory phase  
  
    // sample the Dilithium TSTrajectory (Motion<SE3>) + wrench trajectory  
    Pose3      pose_des = m_pose_traj.evaluate_at(s);  
    SpatialVector3d V_des = m_pose_traj.derivative_at(s, 1);  
    Wrench3d    w_ref    = m_wrench_traj.evaluate_at(s); // JSTrajectory (R^6)  
  
    // feedback: current pose from the model, wrench from the force sensor  
    Pose3      pose_cur = m_model->get_world_H_body(Part::ARM, m_frame_id);  
    Wrench3d w_sensor = read_force_sensor(status, m_force_sensor_name);  
  
    // admittance law - the ONLY law that consumes a wrench  
    SpatialVector3d V = m_Kx_ff * V_des  
        + m_Kx_p * pose_error(pose_des, pose_cur)  
        + m_Kx_f * (w_ref - w_sensor);  
  
    Eigen::VectorXd qvel = resolve_to_joints(V);           // TS twist → joint vel  
    m_cmd.pos += m_dt * qvel;  
  
    cmd.part_commands.at(Part::ARM).set_streaming_command(m_cmd.pos, qvel, {},  
{});  
    if (m_supervisor.trajectory_complete(status))  
        m_state = ControllerState::COMPLETE;  
    return m_state;  
}
```



Do these interfaces meet our criteria?

- **Robust:** The core interfaces haven't changed in a long time
- **Performant:** The most susceptible to performance degradation; this has varied as needs have varied. These interfaces are deterministic and reliable but occasionally rely on libraries that might not be
- **Extensible:** The control interface is one of the least extensible. Because of its interaction with hardware components abstracting at this layer to bring in new libraries is hard
- **Modern readable C++** (with Python Bindings): Yes

Wrapping up

There are 14 standards!



- And that's okay. You do have to form a company standard or group standard and as long as there's a reasonable balance of abstractions it's a good standard

2 minute take away

- Allow for swapping out kinematics/collision backends
- Allow for using the same types in planning, control and simulation where possible
 - Think about how those types would be used in real time and safety critical systems
- Separate out the algorithm from the types where possible
- **We'd like different people to be able to make the choices they would like to make while giving them the flexibility to implement what we need**

Thanks

- **The libraries in this talk are the work of many people — thank you:**
- Alan Sullivan · Alex Kyriazis · Andrew Hopkins · Ankit Bhatia · Challen Enniful Adu · Chandravarun Kunjeti · David Hernandez · Diego Pardo · Fernando Berra Kossik · Gordon Wolfe · James Gadoury · Jeongseok Lee · Jeremy Binagia · Jiangeng Dong · Joseph Sullivan · Josh Davis · Kartik Prakash · Lilly Yu · Mathew Halm · Michael McNeill · Mohit Deshpande · Nathan Kong · Neel Doshi · Nikolay Korolev · Pengcheng Zhao · Petter Nilsson · Priyanka Rao · Rahul Warrier · Ross Hartley · Ruibo Yan · Safi Mohammed · Saikiran Komatineni · Sarah Chen · Silvia Knappe · Srinath Rajagopalan · Tobey Pineda · Vinit Gupta · Wenyu Xia · Xining Du · Yuechuan Xue
- *...and the wider AR Motion & Manipulation team.*
- Standing on open source: Eigen · Pinocchio · Coal (HPP-FCL) · DART · OSQP · NLOpt · Ruckig · ONNX Runtime

Questions?